

Relatório do Trabalho Prático 2: Programação Paralela com CUDA

Aron P. Sarabia, Cassiano C. De Souza

Faculdade de Computação (FACOM)
Universidade Federal do Mato Grosso do Sul (UFMS) – 79070-900
A Av. Costa e Silva, s/nº – Bairro Universitário
Campo Grande – MS – Brasil

{aron_petta, cassiano_c_souza}@ufms.br

Resumo. O presente trabalho foca em apresentar uma resolução ao Método Iterativo de Jacobi através de programação paralela via CUDA. Através de análise de desempenho, verificaremos os principais pontos de reflexão sobre os limites e desafios encontrados na paralelização do algoritmo do método.

1. Introdução

O Método de Jacobi resolve um Sistema Linear através de sucessivas iterações de aproximação à um vetor-solução escolhido previamente. Quando há convergência com a solução, o sistema é interrompido.

A implementação do paralelismo no algoritmo do método inclui fragmentar os processos do laço de interação através do multiprocessamento de threads específicas para otimizar a resolução. Isso permite que uma quantidade massiva de cálculos diferentes sejam processados em paralelo, evitando atrasos e dependências que poderiam comprometer o desempenho do algoritmo. Em contraponto, a técnica promove maior velocidade e precisão na busca pela solução do problema.

2. Implementação

Primeiramente, ainda em tempo de compilação, o código reconhece as variáveis que serão utilizadas através de um template genérico. Isso permite criar as diferentes versões para testes de thread-por-blocos e tamanho da matriz gerada. Em seguida, cria as funções de teste que vão levantar o tempo, erro e as interações da CPU e da GPU.

Logo após, ele define e gera a matriz e os vetores de cada implementação (O tamanho da matriz de vetores-solução é montado ainda dentro da Main em um conjunto de testes pré-definidos).

No próximo passo, ele inicia a implementação sequencial e a implementação paralela do algoritmo.

A função de implementação sequencial inicia gerando uma nova lista de vetores-solução; logo mais, garante que o loop do laço comece e que o erro permaneça acima da tolerância; então chama uma estrutura para guardar os dados da iteração e do erro; e continua as iterações até convergir ou atingir o limite. Durante as iterações, percorre cada linha e coluna através de dois laços (for), para no fim realizar as somas de valores. No fim, calcula também o erro e tempo de execução.

Já para a função paralela, ele antes calcula e define qual thread será relacionada a qual processamento de dados. Então, aloca e transfere dados da memória da CPU para a GPU, mede a quantidade de blocos necessários para a execução e usa o ***jacobi_CUDA<<<blocos, threads>>>*** para chamar a paralelização da função pelo Kernel e ***cudaDeviceSynchronize()*** para sincronizar os dados. Feito isso ele inverte os ponteiros dos vetores para evitar cópia desnecessária dos recursos e

Ao final de toda a execução, os dados são estruturados em uma tabela, relacionados de forma comparativa (estrutural X paralelizado) e printados para o usuário.

3. Ambiente Experimental

- Processador (modelo e frequência): 11th Gen Intel® Core™ i5-1135G7.
- Número de núcleos e threads: 4 Núcleos e 8 Threads.
- Memória RAM: 24,0 GiB
- Sistema operacional: Fedora Linux 43 (Workstation Edition)
- O ambiente foi no google colab:
 - Placa de vídeo: T4 GPU.
 - Compilador e versão:
 - nvcc: NVIDIA (R) Cuda compiler driver
 - Copyright (c) 2005-2025 NVIDIA Corporation
 - Built on Fri_Feb_21_20:23:50_PST_2025
 - Cuda compilation tools, release 12.8, V12.8.93
 - Build cuda_12.8.r12.8/compiler.35583870_0

4. Resultados

4.1. Tempos de execução

Abaixo foram ilustrados em gráficos, separados pelo tamanho, os dados das seguintes matrizes utilizadas: Matriz de 250, Matriz de 500, Matriz de 1000 e Matriz de 1500.

Grafico do Teste de 250

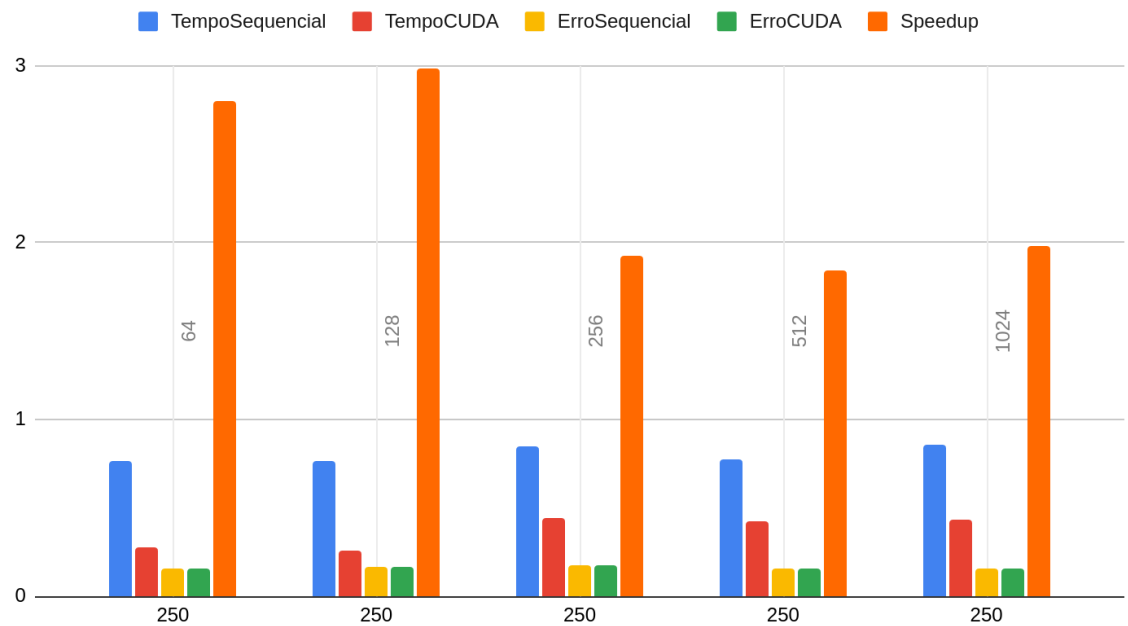


Grafico do Teste de 500

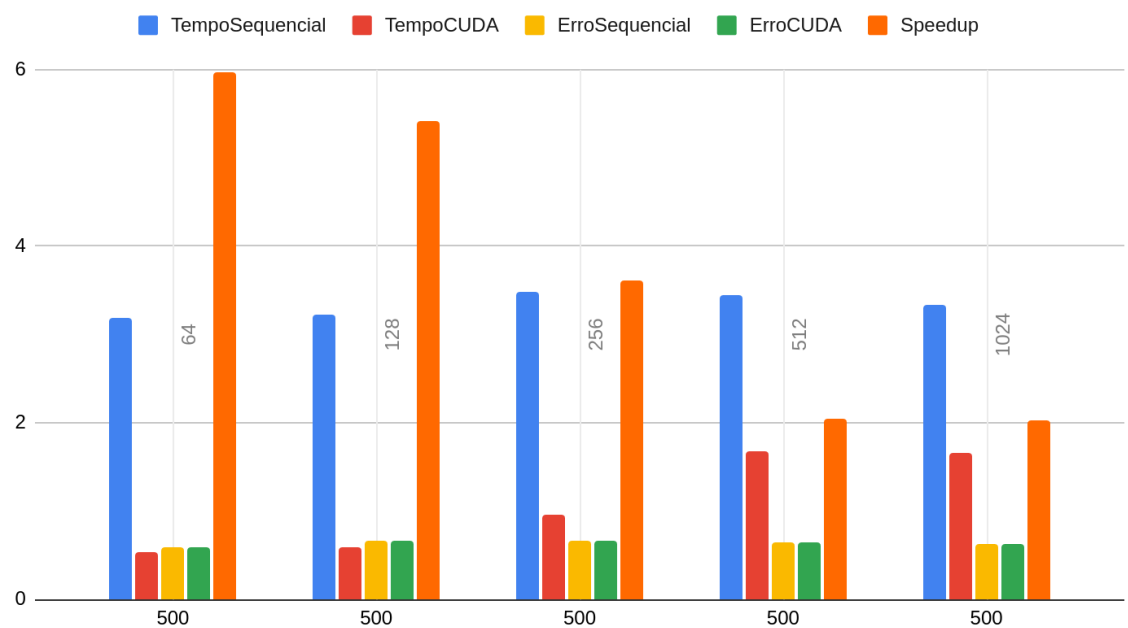


Grafico do Teste de 1500

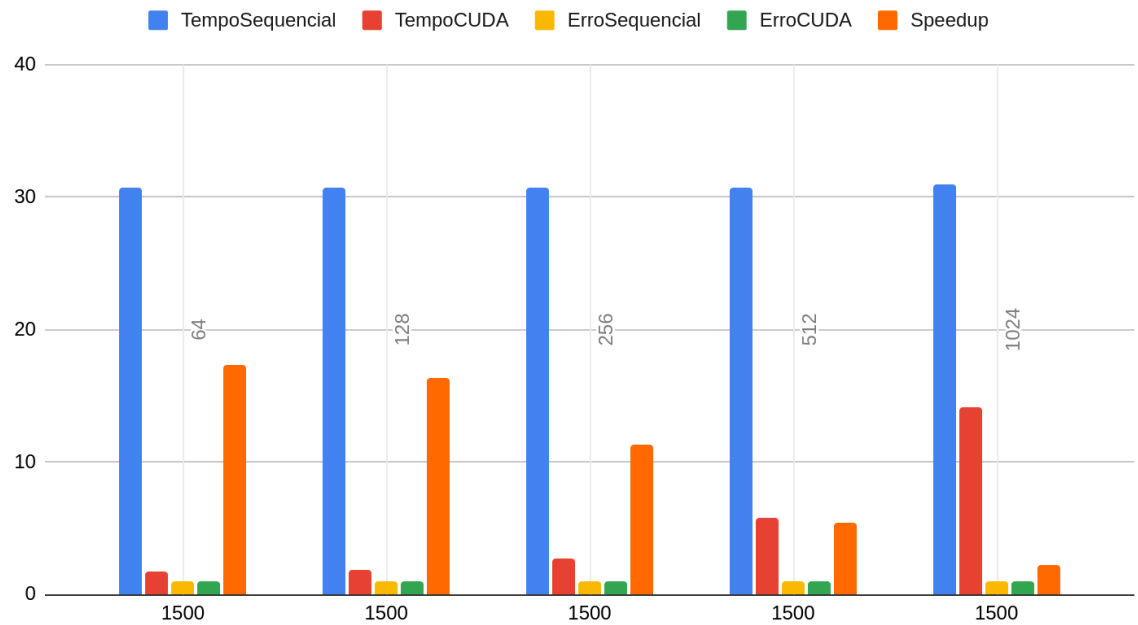
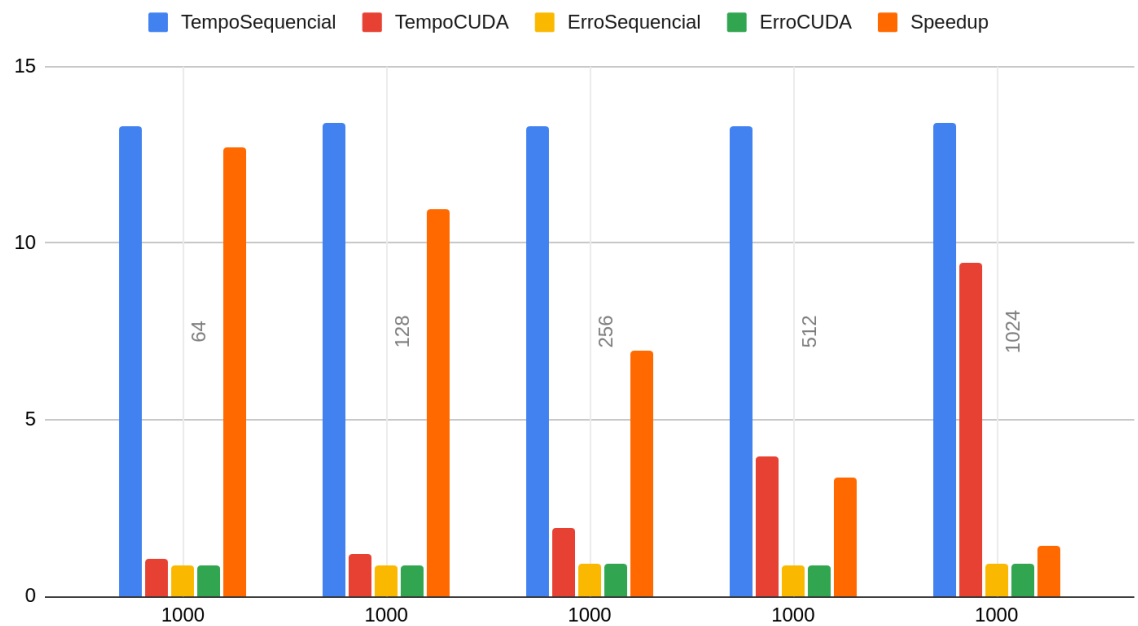


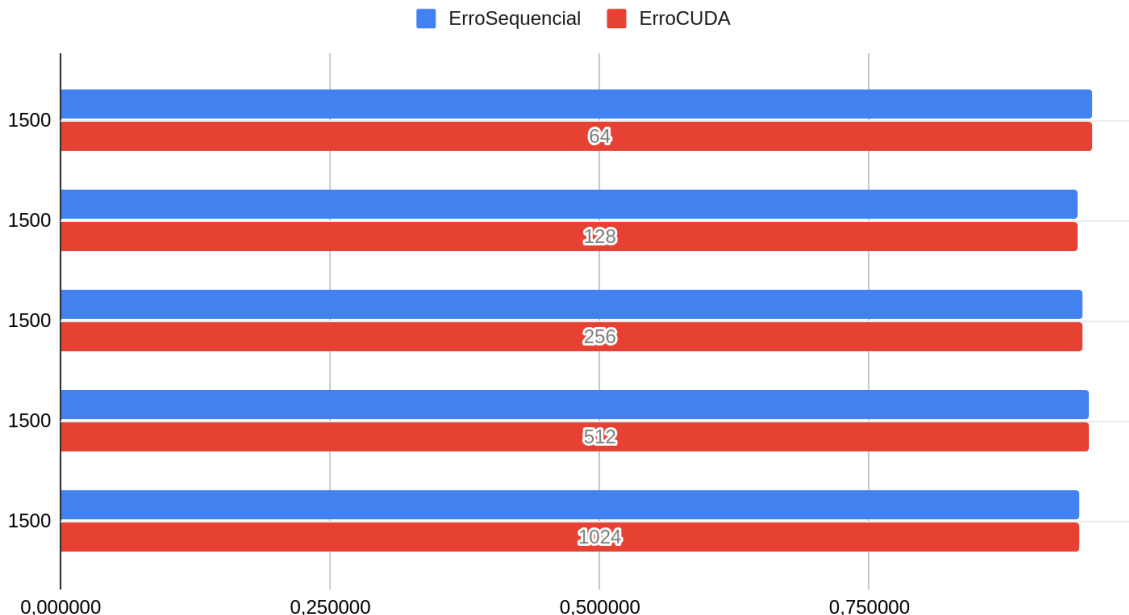
Grafico do Teste de 1000



4.2. Convergência

- O Gráfico abaixo nos mostra que com 10.000 iterações uma matriz de tamanho 1500, não houve tempo o suficiente para convergir para o erro de $1,00E-06$.

Gráfico ERRO de 1500



4.3. Discussão

- A execução paralela do método de Jacobi em GPU (*Device*) exigiu um gerenciamento rigoroso de memória devido às diferenças em relação à CPU (*Host*). Inicialmente, a matriz *A* e os vetores do sistema foram alocados na memória global da GPU via `cudaMalloc()` e os dados foram copiados do *Host* para o *Device* através de `cudaMemcpy()`. Para otimizar a performance, essa transferência via barramento PCIe foi restrita ao início e ao fim do algoritmo, evitando gargalos de comunicação durante o processo iterativo.
- O mapeamento paralelo foi estruturado associando **uma thread exclusiva para cada linha (equação) do sistema**. Com isso, o laço de repetição externo do Jacobi tradicional foi completamente linearizado, permitindo que a GPU processe todas as equações simultaneamente. O controle do número máximo de iterações (`MAX_ITER`) permaneceu na CPU, que gerencia os disparos sucessivos do *kernel* e realiza a inversão ágil dos ponteiros dos vetores (`d_xOld` e `d_xNew`) a cada iteração.
- O foco principal da análise consistiu em variar o tamanho dos blocos de 64 a 1024 threads. Os resultados evidenciaram que blocos

menores (64 threads) geraram maior eficiência para o tamanho do problema testado ($N \leq 1500$). Configurações com 1024 threads por bloco resultaram em queda drástica de desempenho devido à subutilização dos processadores multiprocessadores (SMs) da GPU e ao desperdício de hardware com threads ociosas que falharam na checagem de limite do vetor.

5. Conclusão

A solução paralela, implementada com o algoritmo do Método de Jacobi, supera os problemas da execução sequencial ao distribuir o processamento entre múltiplas threads, tornando a execução mais rápida e dinâmica. Dessa forma, os resultados são obtidos em um tempo consideravelmente menor quando comparados ao sequencial, com um ganho de desempenho evidente. Além disso, também é possível observar crescimento gradual de eficiência conforme os valores aumentam, evidenciando que quanto maior a matriz utilizada nesse método, melhor o desempenho do algoritmo.

Assim, a solução paralela apresenta desempenho geral superior, mantendo tempos reduzidos mesmo em entradas maiores e demonstrando escalabilidade mais consistente. Comparada ao método sequencial, ela se mostra a abordagem mais adequada para aplicações que lidam com grandes volumes de dados.